`secure_val`: a secure-clear-on-move type

Document number: P1315R1 [latest]
Date: 2018-11-26
Author: Miguel Ojeda <miguel@ojeda.io>
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: LWG, LEWG

Abstract

Sensitive data, like passwords or keying data, should be cleared from memory as soon as they are not needed. This requires ensuring the compiler will not optimize the memory overwrite away. This proposal adds a `secure_clear` function to do so as well as a non-copyable `secure_val` class template which represents a memory area which securely clears itself on destruction and move.

The problem

When manipulating sensitive data, like passwords in memory or keying data, there is a need for library and application developers to clear the memory after the data is not needed anymore [1][2][3][4], in order to minimize the time window where it is possible to capture it (e.g. ending in a core dump or probed by a malicious actor). This requires ensuring the compiler will not optimize away the memory write. In particular, for C++, extra care is needed to consider all exceptional return paths.

For instance, the following function may be vulnerable, since the compiler may optimize the `memset` call away because the `password` buffer is not read from before it goes out of scope:

```
void f()
{
    constexpr std::size_t size = 100;
    char password[size];

    getPasswordFromUser(password, size);

    // ...

    usePassword(password, size);
    std::memset(password, size);
}
```

On top of that, `usePassword` could throw an exception (i.e. assume the stack is not overwritten and/or that the memory is held in the free store).

There are many ways that developers may use to try to ensure the memory is cleared as expected (i.e. avoiding the optimizer):

- Calling a function defined in another translation unit, assuming LTO/WPO is not enabled.
- Writing memory through a `volatile` pointer (e.g. `decaf_bzero` [5] from OpenSSL).
- Calling `memset` through a `volatile` function pointer (e.g. `OPENSSL_cleanse` C implementation [6]).
- Creating a dependency by writing an `extern` variable into it (e.g. `CRYPTO_malloc` implementation [7] from OpenSSL).
- Coding the memory write in assembly (e.g. `OPENSSL_cleanse` SPARC implementation [8]).
- Introducing a memory barrier (e.g. `memzero_explicit` implementation [9] from the Linux Kernel).

- Disabling specific compiler options/optimizations (e.g. `-fno-builtin-memset` [10] in GCC).

Or they may use a pre-existing solution whenever available:

- Using an operating system-provided API (e.g. `explicit_bzero` [11] from OpenBSD & FreeBSD, `SecureZeroMemory` [12] from Windows).
- Using a library function (e.g. `memzero_explicit` [13][9] from the Linux Kernel, `OPENSSL_cleanse` [14][6]).

Regardless of how it is done, none of these ways is — at the same time — portable, easy to recognize the intent (and/or `grep` for it), readily available and avoiding compiler implementation details. The topic may generate discussions in major projects on which is the best way to proceed and whether an specific approach ensures that the memory is actually cleansed (e.g. [15][16][17][18][19]). Sometimes, such a way is not effective for a particular compiler (e.g. [20]). In the worst case, bugs happen (e.g. [21][22]).

C11 (and C++17 as it was based on it) added `memset_s` (K.3.7.4.1) to give a standard solution to this problem [4][23][24]. However, it is an optional extension (Annex K) and, at the time of writing, several major compiler vendors do not define `__STDC_LIB_EXT1__` (GCC [25], Clang [26], MSVC [27], icc [28]). Therefore, in practical terms, there is no standard solution yet for C nor C++. A 2015 paper on this topic, WG14's N1967 "Field Experience With Annex K — Bounds Checking Interfaces" [29], concludes that Annex K should be removed from the C standard.

Moreover, while ensuring that the memory is cleared as soon as possible is a good practise, there are other potential improvements when handling sensitive data in memory:

- Reducing the number of copies to a minimum.
- Clearing registers, caches, the entire stack frame, etc.
- Locking/pinning memory to avoid the data being swapped out to external storage (e.g. disk).
- Encrypting the memory while it is not being accessed to avoid plain-text leaks (e.g. to a log or in a core dump).
- Turning off speculative execution (or mitigating it). At the time of writing, Spectre-class vulnerabilities are still being fixed (either in software or in hardware), and new ones are coming up [30].
- Techniques related to control flow, e.g. control flow balancing (making all branches spend the same amount of time).

Most of these extra improvements, however, require either non-portable code or compiler support; which makes them a good target for standardization.

Other languages offer similar facilities in their standard libraries or as external projects:

- The `SecureString` class in .NET Core, .NET Framework and Xamarin [31].
- The `securemem` package in Haskell [32].
- The `secstr` library in Rust [33].
- Limited private types in Ada and SPARK [34][35].

In addition, there are also efforts on compilers to address these issues:

- The SECURE project and GCC (stack erasure implemented as a function attribute) [36][37][38].
- Research on controlling side-effects in mainstream C compilers [39].

A solution

We can provide a simple `secure_clear` function that takes a pointer and a size to replace the `memset` but won't be optimized away (with some extra notes explained later on):

```
std::secure_clear(password, size);
```

And also a `secure_clear` function template that takes any `T&` and clears it entirely:

```
std::secure_clear(password);
```

These solve the basic problem described above. However, we may go further: we can take advantage of move semantics to define an easy-to-use `secure_val` class template which securely handles sensitive values and can be passed around, if needed; but never copied. The class takes care of setting up the right storage for the sensitive data (e.g. locked memory) and clearing it on move and destruction. On top of that, it provides a single-point of access to such data (which may be used to provide extra guarantees, e.g. memory encryption or using special storage).

With this proposal, the aforementioned code could be re-written as:

```
void f()
{
    constexpr std::size_t size = 100;
    std::secure_val<char[size]> password;

    password.write_access([](auto & data) {
        getPasswordFromUser(data, size);
    });

    // ...

    password.read_access([](const auto & data) {
        usePassword(data, size);
    });
}
```

With this relatively simple change the user has:

- Made clear that such value is sensitive.
- Made clear the points where such data is accessed.
- Ensured the data is securely erased.
- Ensured the data won't be copied.
- If the implementor provides additional protections, minimized the risk of leaking the plain-text data (e.g. into a log, a core dump, an external agent, etc.).

Note that making the class template as easy-to-use as possible and hard-to-misuse is critical due to the nature of the code. On this matter, move semantics help a great deal, because they enable us to disallow copying while providing the user with a way to safely move around the data. It also allows the user to write simple code like:

```
const auto password = getPasswordFromUser();
usePassword(std::move(password));
```

Proposal

This proposal suggests adding a new header, `<secure>`, containing a `secure_clear` function, a `secure_clear` function template and a `secure_val` class template.

`secure_clear` function

```
namespace std {
    void secure_clear(void* data, size_t size) noexcept;
}
```

The `secure_clear` function is intended to make the contents of the memory range `[data, data+size)` impossible to recover. It can be considered equivalent to a call to `memset(data, value, size)` (with an unspecified `value`; there may even be different values, e.g. randomized). However, the compiler must guarantee the call is never optimized out unless the data was not in memory to begin with.

- To clarify: the call may be removed by the compiler if there is no actual memory involved, instead of forcing itself to use actual memory and then clearing it (which would make the call pointless and less secure to begin with). For instance, if the compiler chose to keep the data in a register because it is small enough (and its address was not taken apart from the call to `secure_clear`), then the call could be elided. In a way, there was no memory to clear, so it could be considered that it was not optimized out.

In addition, the compiler may provide further guarantees, like clearing other known copies of the data (e.g. in registers or cache).

`secure_clear` function template

```
namespace std {
    template <typename T>
    void secure_clear(T& object) noexcept;
}
```

The `secure_clear` function template is equivalent to a call to `secure_clear(addressof(object), sizeof(object))`.

`secure_val` class template

```
namespace std {
    template <typename T>
    class secure_val
    {
        T value_; // exposition only
        static_assert(is_trivial_v<T>); // exposition only

    public:
        secure_val() noexcept;

        secure_val(const secure_val<T>&) = delete;
        secure_val(secure_val<T>&&) noexcept;

        ~secure_val();

        secure_val<T>& operator=(const secure_val<T>&) = delete;
        secure_val<T>& operator=(secure_val<T>&&) noexcept;

        void clear() noexcept;

        template <typename F>
        void write_access(F f)
noexcept(noexcept(f(std::declval<T&>())));

        template <typename F>
        void read_access(F f) const
noexcept(noexcept(f(std::declval<const T&>())));

        template <typename F>
        void modify_access(F f)
noexcept(noexcept(f(std::declval<T&>())));

        void swap(secure_val<T>&) noexcept;
    }

    template <typename T>
    void swap(secure_val<T>&, secure_val<T>&) noexcept;
}
```

The `secure_val` class template is intended to represent a value which will hold sensitive data (e.g. passwords). In particular, it will provide — at the very least — the following behaviour:

- Securely clears on destruction (as if `secure_clear` was called on the data).
- Securely clears on move (as if `secure_clear` was called on the moved-from data).
- `clear` securely clears on demand (as if `secure_clear` was called on the data).
- `write_access` provides write access to the data during `f` invokation, passing a `T&`.
    - Read access to the previously stored data is not provided, i.e. the user has to assume the original value is lost and the provided one is uninitialized and cannot be read from.
    - The previously stored data is securely cleared, if needed (as if `secure_clear` was called on the data).
    - For instance, if the class implements memory encryption, the class may invoke `f` with a reference to the underlying value (which may contain encrypted data, which the user shouldn't attempt to read) and, on return, encrypt it. It may not be deemed neccesary to securely clear the previous value before `f` invokation, since it will be overwritten.
- `read_access` provides read access to the data during `f` invokation, passing a `const T&`.
    - Any temporary values are securely cleared, if needed (as if `secure_clear` was called on them).
    - For instance, if the class implements memory encryption, the class may decrypt the memory into a temporary buffer, then invoke `f` with a `const` reference to it, and, on return, securely clear it.
- `modify_access` provides access to the data in a read-modify-write cycle during `f` invokation, passing a `T&`.
    - The previously stored data is securely cleared, if needed (as if `secure_clear` was called on the data).
    - Any temporary values are securely cleared, if needed (as if `secure_clear` was called on them).
    - For instance, if the class implements memory encryption, the class may decrypt the memory into a temporary buffer, then invoke `f` with a reference to it, and, on return, encrypt it again into the underlying data and securely clear the temporary (note: in contrast with `write_access`, which may not need to securely clear the data since the memory will be overwritten).

The compiler/implementor may provide further guarantees, like:

- Locking/pinning the memory to avoid being swapped out to external storage (e.g. disk).
- Encrypting the data in memory and only providing a decrypted temporary copy during `read_access` and `modify_access` invokation (and encrypting it in `write_access` and `modify_access`).
    - Even better, providing encryption "on-the-fly", i.e. encrypting/decrypting the smallest amount of data possible (e.g. in the extreme, `char`-by-`char`); although it would require compiler support or a wrapper to the passed `T&` value.
- Avoiding memory to begin with, e.g. by:
    - Keeping the data in registers instead of main memory.
    - Using special "storage" to keep the data in (e.g. special memory in a given system, a secure enclave, etc.).
- Disable speculative execution or mitigate potential Spectre-class vulnerabilities during the `*_access` invokations.

Requirements:

- `T` must be a `TrivialType`. This is intended to allow the compiler to treat the data as a simple memory block which can be easily managed, copied, encrypted, etc. and in order to provide any further guarantees.
- `F` (template parameter of the `*_access` member templates) must be `Callable` with a single parameter `T&` (for `write_access` and `modify_access`) and `const T&` (for `read_access`).

Possible misusages

A particular concern is that developers may try to create types like `secure_val<string>` (i.e. where the actual data is not protected). While it is hard to prevent all misusage, at least the particular case of `secure_val<string>` (and similar third-party `string`-like classes) is not allowed since `string` is not a `TrivialType`.

Alternatives

Removing `secure_clear` from this proposal (i.e. only providing `secure_val`).

- The function is provided so that developers have portable and standard access to the most basic primitive required to clear memory securely.
- It may be also useful to implement the `secure_val` class template itself.

Instead of providing `secure_clear`, this proposal could make `memset_s` from C11 to be required in the C++ standard instead.

Removing `secure_val` from this proposal (i.e. only providing `secure_clear`).

- As long as developers have access to the `secure_clear` primitive, they can implement other solutions or write `secure_val` themselves (a subset of it). However, providing it with the standard library could make its use more widespread.
- Compiler writers may be able to provide more guarantees with `secure_val`.

Possible extensions

Other related functionality could be proposed under the same header in the future:

- A dynamically-sized string `secure_string` could be considered useful.
- A way to read non-echoed standard input could be added, like a `get_password` function taking a `secure_val<char [N]>` by reference (see, for instance, the `getpass` module in Python [40]).

Naming

Several alternatives were considered for the prefix of `secure_clear` and `secure_val` as well as the name of the header:

- `explicit`: follows `explicit_bzero` from OpenBSD & FreeBSD [11]. It collides with `explicit` (the keyword). It does not make for a good header name nor a good class template name. Only `explicit_clear` is a good choice.
- `sensitive`: follows a common word used in this context. `sensitive_val` is a very clear name. However, `sensitive_clear` does not look like an optimal choice.
- `secure`: follows `SecureZeroMemory` from Windows [12]. It is a good compromise and makes the purpose reasonably clear, for both the class template and the function. It may also allow the header to be more general (i.e. used to other security-related definitions).

Several alternatives were considered for the second part of the name of the `secure_clear` function:

- `zero`: follows `explicit_bzero` from OpenBSD & FreeBSD [11] and `SecureZeroMemory` from Windows [12]. Unambiguous, but the fact that the

memory is cleared using the `0` value could be thought as an implementation detail.
- `set`: follows `memset_s` from C11. The name seems to suggest that a value is required (i.e. the value to overwrite memory with), which should not matter.
- `clear`: follows some C++ standard library member function (e.g. like those of the containers). It is more general: a verb commonly used with memory to refer to its deletion.

Several alternatives were considered for the second part of the name of the `secure_val` class template. See the `unique_val` proposal for a discussion on it [41].

Example implementation

A trivial example implementation (i.e. without further compiler guarantees nor memory encryption) can be found at [42].

Acknowledgements

Thanks to Martin Sebor for pointing out the SECURE project talk at the GNU Tools Cauldron 2018.

References

1. MSC06-C. Beware of compiler optimizations — *https://wiki.sei.cmu.edu/confluence/display/c/MSC06-C.+Beware+of+compiler+optimizations*
2. CWE-14: Compiler Removal of Code to Clear Buffers — *https://cwe.mitre.org/data/definitions/14.html*
3. V597. The compiler could delete the `memset` function call (…) — *https://www.viva64.com/en/w/v597/*
4. N1381 — #5 `memset_s()` to clear memory, without fear of removal — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1381.pdf*
5. `openssl/crypto/ec/curve448/utils.c` (old code) — *https://github.com/openssl/openssl/blob/f8385b0fc0215b378b61891582b0579659d0b9f4/crypto/ec/curve448/utils.c*
6. `OPENSSL_cleanse` (implementation) — *https://github.com/openssl/openssl/blob/master/crypto/mem_clr.c*
7. `openssl/crypto/mem.c` (old code) — *https://github.com/openssl/openssl/blob/104ce8a9f02d250dd43c255eb7b8747e81b29422/crypto/mem.c#L143*
8. `openssl/crypto/sparccpuid.S` (example of assembly implementation) — *https://github.com/openssl/openssl/blob/master/crypto/sparccpuid.S#L363*
9. `memzero_explicit` (implementation) — *https://elixir.bootlin.com/linux/v4.18.5/source/lib/string.c#L706*
10. Options Controlling C Dialect — *https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html*
11. `bzero`, `explicit_bzero` - zero a byte string — *http://man7.org/linux/man-pages/man3/bzero.3.html*
12. `SecureZeroMemory` function — *https://msdn.microsoft.com/en-us/library/windows/desktop/aa366877(v=vs.85).aspx*
13. `memzero_explicit` — *https://www.kernel.org/doc/htmldocs/kernel-api/API-memzero-explicit.html*
14. `OPENSSL_cleanse` — *https://www.openssl.org/docs/man1.1.1/man3/OPENSSL_cleanse.html*
15. Reimplement non-asm `OPENSSL_cleanse()` #455 — *https://github.com/openssl/openssl/pull/455*
16. How to zero a buffer — *http://www.daemonology.net/blog/2014-09-04-how-to-zero-a-buffer.html*
17. Hacker News: How to zero a buffer (daemonology.net) — *https://news.ycombinator.com/item?id=8270136*
18. Mac OS X equivalent of `SecureZeroMemory` / `RtlSecureZeroMemory`? — *https://stackoverflow.com/questions/13299420/*

19. Optimising away `memset()` calls? — *https://gcc.gnu.org/ml/gcc-help/2014-10/msg00047.html*
20. Bug 15495 - dead store pass ignores memory clobbering asm statement — *https://bugs.llvm.org/show_bug.cgi?id=15495*
21. Bug 82041 - `memset` optimized out in `random.c` — *https://bugzilla.kernel.org/show_bug.cgi?id=82041*
22. [PATCH] random: add and use memzero_explicit() for clearing data — *https://lkml.org/lkml/2014/8/25/497*
23. N1548 — C11 draft — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1548.pdf*
24. `memset`, `memset_s` — *https://en.cppreference.com/w/c/string/byte/memset*
25. Test for `memset_s` in gcc 8.2 at Godbolt — *https://godbolt.org/g/M7MyRg*
26. Test for `memset_s` in clang 6.0.0 at Godbolt — *https://godbolt.org/g/ZwbkgY*
27. Test for `memset_s` in MSVC 19 2017 at Godbolt — *https://godbolt.org/g/FtrVJ8*
28. Test for `memset_s` in icc 18.0.0 at Godbolt — *https://godbolt.org/g/vHZNrW*
29. N1967 (WG14) — Field Experience With Annex K - Bounds Checking Interfaces — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1967.htm*
30. CppCon 2018: Chandler Carruth “Spectre: Secrets, Side-Channels, Sandboxes, and Security” — *https://www.youtube.com/watch?v=_f7O3IfIR2k*
31. `SecureString` Class — *https://docs.microsoft.com/en-us/dotnet/api/system.security.securestring*
32. `securemem`: abstraction to an auto scrubbing and const time eq, memory chunk. — *https://hackage.haskell.org/package/securemem*
33. `secstr`: Secure string library for Rust — *https://github.com/myfreeweb/secstr*
34. Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…) — *https://proteancode.com/wp-content/uploads/2017/06/ae2017_final3_for_web.pdf*
35. Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…) — *https://link.springer.com/chapter/10.1007%2F978-3-319-60588-3_3*
36. The SECURE Project and GCC - GNU Tools Cauldron 2018 — *https://www.youtube.com/watch?v=Lg_jJLtH7R0*
37. The SECURE project and GCC — *https://gcc.gnu.org/wiki/cauldron2018#secure*
38. The Security Enhancing Compilation for Use in Real Environments (SECURE) project — *https://www.embecosm.com/research/secure/*
39. What you get is what you C: Controlling side effects in mainstream C compilers — *https://www.cl.cam.ac.uk/~rja14/Papers/whatyouc.pdf*
40. Portable password input — *https://docs.python.org/3.7/library/getpass.html*
41. `unique_val`: a default-on-move type — *https://ojeda.io/cpp/unique_val*
42. `secure_val` Example implementation — *https://github.com/ojeda/secure_val/tree/master/proposal*